

ENRIQUE CRESPO RAMIREZ

ARQUITECTURA EN LA NUBE – ADMINISTRACION REMOTA DE
SERVIDORES WEB EN AWS A TRAVES DE SSH

PARTE 1

Verificamos la versión WSL que tenemos instalada

```
C:\Windows\System32>wsl --version
Versión de WSL: 2.6.1.0
Versión de kernel: 6.6.87.2-1
Versión de WSLg: 1.0.66
Versión de MSRDC: 1.2.6353
Versión de Direct3D: 1.611.1-81528511
Versión de DXCore: 10.0.26100.1-240331-1435.ge-release
Versión de Windows: 10.0.26100.5074
```

Comprobamos la versión WSL

Creamos el directorio para las claves SSH

Queremos un espacio seguro dentro de tu entorno Linux (WSL) donde guardar las claves privadas (.pem) que usaremos para acceder a servidores AWS mediante SSH. SSH exige permisos muy restrictivos para esas claves (nadie más debe poder leerlas).

```
kike@A6Alumno09:/mnt/c/Windows/System32$ mkdir -p ~/.ssh
kike@A6Alumno09:/mnt/c/Windows/System32$ mkdir -p ~/.ssh
chmod 700 ~/.ssh
```

Creación el directorio

Explicación detallada

◆ `mkdir -p ~/.ssh`

Crea el directorio oculto `.ssh` en tu carpeta de usuario.

El parámetro `-p` evita errores si ya existiera.

Ruta resultante: `/home/tu_usuario/.ssh`

◆ `chmod 700 ~/.ssh`

Cambia los permisos del directorio:

- `7` → el propietario (tú) puede leer, escribir y ejecutar.
- `0` y `0` → ningún otro usuario tiene acceso.

Esto es importante porque si el directorio tuviera permisos demasiado abiertos, SSH rechazaría usar las claves privadas.

Explicación técnica de comandos

Comprobamos los permisos y la existencia del directorio con:

`ls -ld ~/.ssh`

```
kike@A6Alumno09:/mnt/c/Windows/System32$ ls -l ~/.ssh/  
total 4  
-r----- 1 kike kike 1674 Nov 12 10:51 labsuser.pem
```

Ahora Vamos a mover la clave que descargamos de AWS desde nuestra carpeta de Windows a tu entorno WSL.



Clave descargada de AWS

Abrimos la terminal de WSL y ejecutamos el siguiente comando

`cp /mnt/c/Users/Alumno.DESKTOP-DI5KTUG/Downloads/labsuser.pem ~/.ssh/`

Luego le damos permisos

`chmod 400 ~/.ssh/labsuser.pem`

Verificamos que se copio correctamente

```
ls -l ~/.ssh/
```

```
kike@A6Alumno09:/mnt/c/Windows/System32$ ls -l ~/.ssh/
total 4
-rwxr-xr-x 1 kike kike 1674 Nov 12 10:51 labsuser.pem
kike@A6Alumno09:/mnt/c/Windows/System32$
```

Permisos correctos clave segura y lista para usarse por ssh

Creación de instancia EC2

Esto lo hicimos en clase, así que adjunto la descripción de la instancia.

Resumen de instancia de i-024f982a2d93ffbd0 (Mi-Servidor-Web) Información

Se ha actualizado hace less than a minute

ID de la instancia

i-024f982a2d93ffbd0

Dirección IPv6

-

Tipo de nombre de anfitrión

Nombre de IP: ip-172-31-16-178.ec2.internal

Responder al nombre DNS de recurso privado

IPv4 (A)

Dirección IP asignada automáticamente

44.220.149.12 [IP pública]

Rol de IAM

-

IMDSv2

Required

Operador

-

Dirección IPv4 pública

44.220.149.12 | [dirección abierta](#)

Estado de la instancia

En ejecución

Nombre DNS de IP privada (solo IPv4)

ip-172-31-16-178.ec2.internal

Tipo de instancia

t3.micro

ID de VPC

vpc-041d538f2529bac81

ID de subred

subnet-06a91a42d3a4b010b

ARN de instancia

arn:aws:ec2:us-east-1:722622494352:instance/i-024f982a2d93ffbd0

Direcciones IPv4 privadas

172.31.16.178

DNS público

ec2-44-220-149-12.compute-1.amazonaws.com | [dirección abierta](#)

Direcciones IP elásticas

-

Hallazgo de AWS Compute Optimizer

[Suscribirse a AWS Compute Optimizer para recibir recomendaciones.](#) | [Más información](#)

Nombre del grupo de Auto Scaling

-

Administradas

falso

Descripción de la instancia EC2.

Configurar las reglas de entrada

	Name	ID de la regla del gr...	Versión de IP	Tipo	Protocolo	Intervalo de puertos	Origen	De
☐	Nginx	sgr-0ba2effea127560a7	IPv4	TCP personalizado	TCP	8081	0.0.0.0/0	-
☐	Apache HTTPS (SSL)	sgr-0f45439b1c45037ea	IPv4	TCP personalizado	TCP	8443	0.0.0.0/0	-
☐	Caddy	sgr-0cc15f83ca2509706	IPv4	TCP personalizado	TCP	8082	0.0.0.0/0	-
☐	Apache HTTP	sgr-060bbe29a1d063f76	IPv4	TCP personalizado	TCP	8080	0.0.0.0/0	-
☐	Acceso SSH Remoto	sgr-08bf6ad65b2fd2ec9	IPv4	SSH	TCP	22	0.0.0.0/0	-

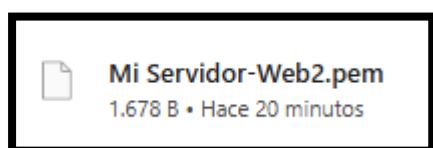
Tabla descriptiva de reglas de entrada

Especificar la clave PEM al lanzar la instancia

Ahora estamos diciéndole a AWS **qué clave privada (PEM)** vamos a usar para conectarte al servidor cuando esté encendido.

AWS guarda la **clave pública** correspondiente dentro de la instancia, y tú usas la **clave privada** (labsuser.pem) desde tu máquina para autenticarte por SSH.

Si no seleccionas la clave correcta, **no podrás acceder** a la instancia después.

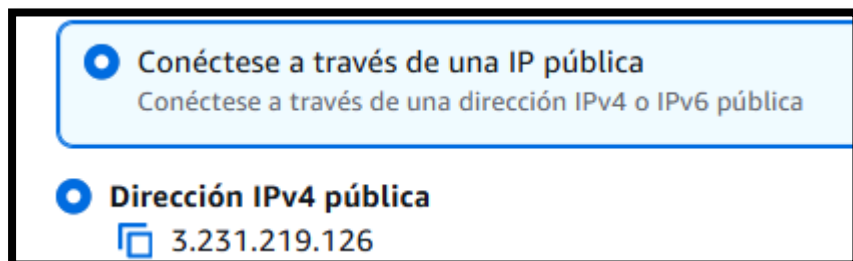


Clave .Pem

PARTE 2

Conexion SSH DESDE WSL A AWS

Obtenemos la IP publica de la instancia



IPV4 Publica

Ejecutamos `ssh -i ~/.ssh/mi-servidor-web2.pem ubuntu@3.231.219.126`

```
kike@A6Alumno09:~/.ssh$ ssh -i ~/.ssh/mi-servidor-web2.pem ubuntu@3.231.219.126
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1015-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Nov 13 09:11:36 UTC 2025

System load:  0.0               Temperature:   -273.1 C
Usage of /:   26.2% of 6.71GB   Processes:    114
Memory usage: 22%              Users logged in: 1
Swap usage:  0%               IPv4 address for ens5: 172.31.76.39

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Thu Nov 13 08:42:11 2025 from 18.206.107.29
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
```

Ya estamos conectados por SSH

Este comando establece una conexión SSH segura desde WSL hacia el servidor Ubuntu alojado en AWS.

- `ssh` inicia el cliente SSH, que permite conectarse a sistemas remotos mediante un canal cifrado.
- La opción `-i` especifica la clave privada que se utiliza para autenticarse. En este caso, se emplea la clave `mi-servidor-web2.pem`, almacenada en el directorio seguro `~/.ssh`.
- `ubuntu@3.231.219.126` indica el usuario con el que se realizará la conexión (ubuntu, usuario por defecto en las AMI Ubuntu) y la dirección IP pública del servidor.

Posteriormente le vamos a dar permisos con el comando **chmod 400 ~/.ssh/mi-servidor-web2.pem**

```
known_hosts labsuser.pem mi-servidor-web2.pem
kike@A6Alumno09:~/.ssh$ chmod 400 ~/.ssh/mi-servidor-web2.pem
kike@A6Alumno09:~/.ssh$
```

Este comando modifica los permisos del archivo para que **únicamente el propietario tenga permiso de lectura**, y el resto de usuarios no tenga ningún tipo de acceso. Esto evita que terceros puedan copiar o usar la clave privada y es un requisito obligatorio para que el cliente SSH acepte la clave al iniciar la conexión.

```
ubuntu@ip-172-31-76-39:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP group default qlen 1000
    link/ether 16:ff:fe:54:63:e1 brd ff:ff:ff:ff:ff:ff
    altname enp0s5
    inet 172.31.76.39/20 metric 100 brd 172.31.79.255 scope global dynamic ens5
        valid_lft 3067sec preferred_lft 3067sec
    inet6 fe80::14ff:feff:fe54:63e1/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@ip-172-31-76-39:~$
```

Configuracion de Red de Nuestra máquina Virtual en AWS

PARTE 3

instalar apache y moverlo al puerto 8080

Ejecutamos para actualizar repositorios e instalar apache:

sudo apt update

sudo apt install apache2 -y

```
ubuntu@ip-172-31-76-39:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-11-20 08:18:49 UTC; 1h 7min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 520 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Process: 939 ExecReload=/usr/sbin/apachectl graceful (code=exited, status=0/SUCCESS)
  Main PID: 559 (apache2)
    Tasks: 55 (limit: 1017)
   Memory: 10.2M (peak: 12.2M)
      CPU: 331ms
   CGroup: /system.slice/apache2.service
           └─559 /usr/sbin/apache2 -k start
             └─978 /usr/sbin/apache2 -k start
               └─979 /usr/sbin/apache2 -k start

Nov 20 08:18:49 ip-172-31-76-39 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Nov 20 08:18:49 ip-172-31-76-39 systemd[1]: Started apache2.service - The Apache HTTP Server.
Nov 20 08:18:50 ip-172-31-76-39 systemd[1]: Reloading apache2.service - The Apache HTTP Server...
Nov 20 08:18:50 ip-172-31-76-39 systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
ubuntu@ip-172-31-76-39:~$
```

Apache corriendo

Ahora vamos a cambiar Apache al puerto 8080, ejecutamos **sudo nano**
/etc/apache2/ports.conf

```
# /etc/apache2/sites-enabled/000-default.conf

Listen 8080

<IfModule ssl_module>
    Listen 443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 443
</IfModule>
```

Apache funcionando en puerto 8080

Ahora procedemos a cambiar el virtual host, ejecutamos **sudo nano**
/etc/apache2/sites-available/000-default.conf


```

GNU nano 7.2
<VirtualHost *:8080>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
    # specifies what hostname must appear in the request's Host: header to
    # match this virtual host. For the default virtual host (this file) this
    # value is not decisive as it is used as a last resort host regardless.
    # However, you must set it for any further virtual host explicitly.
    #ServerName www.example.com

    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf
</VirtualHost>

```

Virtual Box Modificado al 8080

¿Qué es un VirtualHost en Apache?

En Apache, un VirtualHost es un bloque de configuración que le dice al servidor:

- qué sitio web debe atender
- en qué puerto
- qué carpeta usa como página web
- qué logs escribe

En resumen:

- ◆ Es la configuración de un sitio web dentro de Apache.

Apache puede tener 1, 10 o 100 sitios, cada uno definido en un VirtualHost.

Explicación Técnica VirtualHost en Apache

Ahora comprobamos que está escuchando el puerto 8080 con **sudo ss -tlnp | grep 8080**

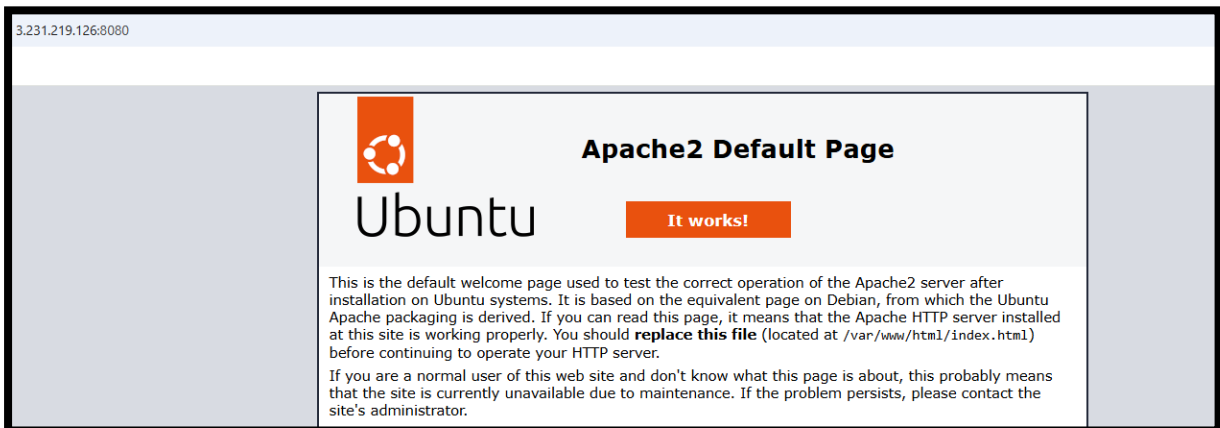
```

Nov 13 09:21:01 ip-172-31-76-39 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Nov 13 09:21:02 ip-172-31-76-39 systemd[1]: Started apache2.service - The Apache HTTP Server.
ubuntu@ip-172-31-76-39:~$ sudo nano /etc/apache2/ports.conf
ubuntu@ip-172-31-76-39:~$ sudo nano /etc/apache2/sites-available/000-default.conf
ubuntu@ip-172-31-76-39:~$ sudo nano /etc/apache2/ports.conf
ubuntu@ip-172-31-76-39:~$ sudo nano /etc/apache2/sites-available/000-default.conf
ubuntu@ip-172-31-76-39:~$ sudo nano /etc/apache2/ports.conf
ubuntu@ip-172-31-76-39:~$ sudo nano /etc/apache2/sites-available/000-default.conf
ubuntu@ip-172-31-76-39:~$ sudo systemctl restart apache2
ubuntu@ip-172-31-76-39:~$ sudo ss -tlnp | grep 8080
LISTEN 0      511          *:8080        *:8080        users:((("apache2",pid=3647,fd=4),("apache2",pid=3646,fd=4),("apache2",pid=3644,fd=4)))
ubuntu@ip-172-31-76-39:~$

```

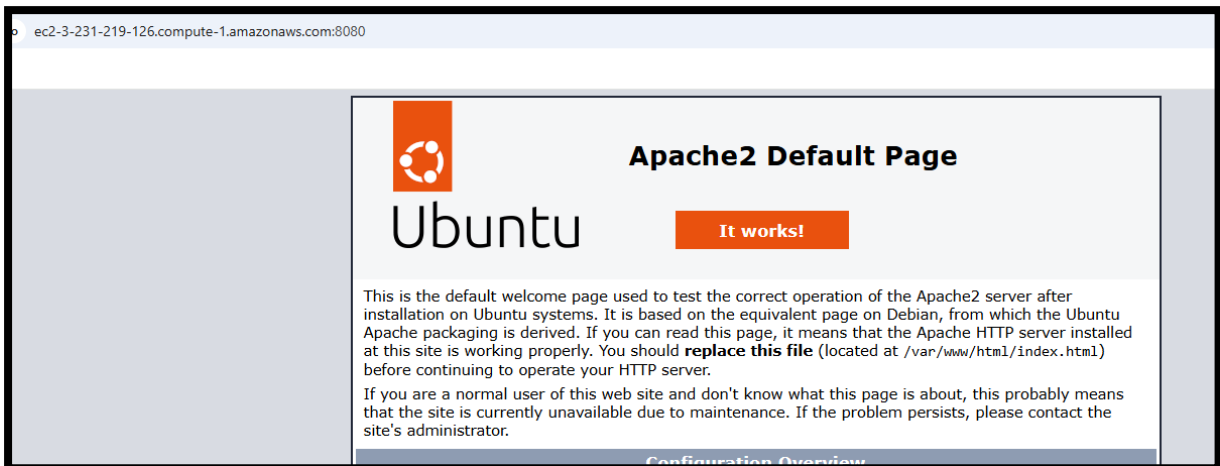
Apache escuchando 8080

Ahora comprobamos por IP en el navegador <http://3.231.219.126:8080>



Apache funcionando por la IP Publica de nuestra máquina virtual de Aws

También podemos comprobarlo por el Dns de AWS con <http://ec2-3-231-219-126.compute-1.amazonaws.com:8080/>



Apache funcionando por DNS AWS

PASO 2 — Instalar y configurar NGINX en el puerto 8081

En primer lugar, instalamos NGINX con **sudo apt install nginx -y**

```
o VM guests are running outdated hypervisor (qemu) binaries on this host.
buntu@ip-172-31-76-39:~$ systemctl status nginx
nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-11-13 09:53:48 UTC; 15s ago
     Docs: man:nginx(8)
   Process: 3945 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Process: 3947 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 3982 (nginx)
    Tasks: 3 (limit: 1008)
   Memory: 2.4M (peak: 5.3M)
      CPU: 26ms
   CGroup: /system.slice/nginx.service
           └─3982 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─3984 "nginx: worker process"
               └─3985 "nginx: worker process"

ov 13 09:53:48 ip-172-31-76-39 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
ov 13 09:53:48 ip-172-31-76-39 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
```

NGINX Funcionando correctamente

Posteriormente cambiamos el puerto de Nginx al 8081 con el comando **sudo nano /etc/nginx/sites-available/default**

```
# Please see /usr/share/doc/nginx-doc/examples/ for more det
##

# Default server configuration
#
server {
    listen 8081 default_server;
    listen [::]:8081 default_server;

    # SSL configuration
    #
    # listen 443 ssl default_server;
    # listen [::]:443 ssl default_server;
    #
    # Note: You should disable gzip for SSL traffic.
    # See: https://bugs.debian.org/773332
```

Modificamos el puerto por el 8081

Ahora comprobamos que está escuchando el puerto correcto **sudo ss -tlnp | grep 8081**

```

Docs: man:nginx(8)
Process: 3945 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
Process: 3947 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
Main PID: 3982 (nginx)
Tasks: 3 (limit: 1008)
Memory: 2.4M (peak: 5.3M)
CPU: 26ms
CGroup: /system.slice/nginx.service
└─3982 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
   └─3984 "nginx: worker process"
      └─3985 "nginx: worker process"

Nov 13 09:53:48 ip-172-31-76-39 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Nov 13 09:53:48 ip-172-31-76-39 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-76-39:~$ sudo nano /etc/nginx/sites-available/default
ubuntu@ip-172-31-76-39:~$ sudo systemctl restart nginx
ubuntu@ip-172-31-76-39:~$ sudo ss -tlnp | grep 8081
LISTEN 0      511          0.0.0.0:8081      0.0.0.0:*        users:(("nginx",pid=4079,fd=5),("nginx",pid=4078,fd=5),("nginx",pid=4077,fd=5))
LISTEN 0      511          [::]:8081        [::]:*           users:(("nginx",pid=4079,fd=6),("nginx",pid=4078,fd=6),("nginx",pid=4077,fd=6))
ubuntu@ip-172-31-76-39:~$

```

Nginx escuchando puerto 8081

```

ubuntu@ip-172-31-76-39:~$ curl -I http://localhost:8081
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Thu, 13 Nov 2025 11:35:27 GMT
Content-Type: text/html
Content-Length: 10671
Last-Modified: Thu, 13 Nov 2025 09:20:59 GMT
Connection: keep-alive
ETag: "6915a2fb-29af"
Accept-Ranges: bytes

ubuntu@ip-172-31-76-39:~$ curl -I http://localhost:8080
HTTP/1.1 200 OK
Date: Thu, 13 Nov 2025 11:35:31 GMT
Server: Apache/2.4.58 (Ubuntu)
Last-Modified: Thu, 13 Nov 2025 09:20:59 GMT
ETag: "29af-64376662c8839"
Accept-Ranges: bytes
Content-Length: 10671
Vary: Accept-Encoding
Content-Type: text/html

```

Comprobacion Apache escuchando 8080 Nginx 8081

PASO 4 Instalar y configurar Caddy en el puerto 8082

Ejecutamos:

sudo apt update

sudo apt install -y caddy

```

buntu@ip-172-31-76-39:~$ systemctl status caddy
caddy.service - Caddy
Loaded: loaded (/usr/lib/systemd/system/caddy.service; enabled; preset: enabled)
Active: active (running) since Thu 2025-11-13 11:39:34 UTC; 11s ago
Docs: https://caddyserver.com/docs/
Main PID: 4593 (caddy)
Tasks: 7 (limit: 1008)
Memory: 6.8M (peak: 7.0M)
CPU: 29ms
CGroup: /system.slice/caddy.service
└─4593 /usr/bin/caddy run --environ --config /etc/caddy/Caddyfile

ov 13 11:39:34 ip-172-31-76-39 caddy[4593]: {"level":"info","ts":1763033974.132745,"msg":"using provided configuration","conf
ov 13 11:39:34 ip-172-31-76-39 caddy[4593]: {"level":"info","ts":1763033974.1351566,"logger":"admin","msg":"admin endpoint st
ov 13 11:39:34 ip-172-31-76-39 caddy[4593]: {"level":"warn","ts":1763033974.1352828,"logger":"http","msg":"server is listenin
ov 13 11:39:34 ip-172-31-76-39 caddy[4593]: {"level":"info","ts":1763033974.135402,"logger":"http_log","msg":"server running"}

```

Comprobacion Caddy funcionando correctamente

Cambiar Caddy al puerto 8082

Ejecutamos `sudo nano /etc/caddy/Caddyfile` , y cambiamos el puerto de Caddy

```

GNU nano 7.2 /etc/caddy/Caddyfile *
The Caddyfile is an easy way to configure your Caddy web server.

Unless the file starts with a global options block, the first
uncommented line is always the address of your site.

To use your own domain name (with automatic HTTPS), first make
sure your domain's A/AAAA DNS records are properly pointed to
this machine's public IP, then replace ":80" below with your
domain name.

8082 {
    # Set this path to your site's directory.
    root * /usr/share/caddy

    # Enable the static file server.
    file_server

    # Another common task is to set up a reverse proxy:
    # reverse_proxy localhost:8080

    # Or serve a PHP site through php-fpm:
    # php_fastcgi localhost:9000

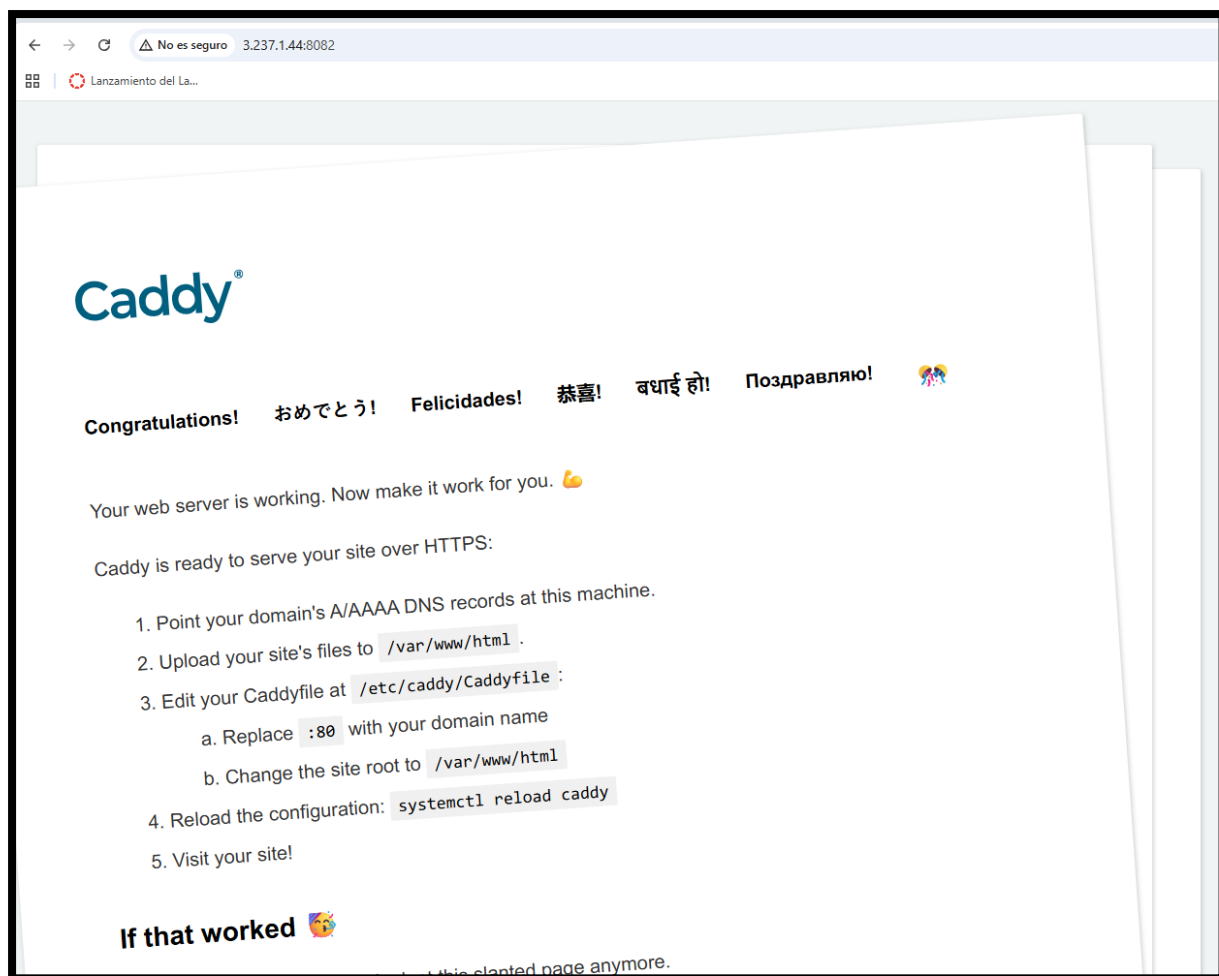
Refer to the Caddy docs for more information:
https://caddyserver.com/docs/caddyfile

```

Archivo de configuración de Caddy

Ahora comprobamos en el navegador que Caddy está funcionando correctamente con

<http://3.237.1.44:8082>



Caddy funcionando correctamente

Instalar dependencias

```
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https curl
```

Agregar repositorio oficial de Caddy

Añadir la clave GPG:

```
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' \
```

```
| sudo gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
```

Añadir el repositorio:

```
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' \
```

```
| sudo tee /etc/apt/sources.list.d/caddy-stable.list
```

Posteriormente vemos el contenido del archivo con **/etc/apt/sources.list.d/caddy-stable.list**

```
ubuntu@ip-172-31-76-39:~$ cat /etc/apt/sources.list.d/caddy-stable.list
# Source: Caddy
# Site: https://github.com/caddyserver/caddy
# Repository: Caddy / stable
# Description: Fast, multi-platform web server with automatic HTTPS

deb [signed-by=/usr/share/keyrings/caddy-stable-archive-keyring.gpg] https://dl.cloudsmith.io/public/caddy/stable/deb/debian any-version main
deb-src [signed-by=/usr/share/keyrings/caddy-stable-archive-keyring.gpg] https://dl.cloudsmith.io/public/caddy/stable/deb/debian any-version main
ubuntu@ip-172-31-76-39:~$
```

Registro de el repositorio oficial de Caddy

Crear directorio para Caddy

Ejecutamos:

```
sudo mkdir -p /var/www/caddy
```

Crear el archivo Markdown de prueba

Ejecutamos:

```
echo "# Bienvenido a Caddy" | sudo tee /var/www/caddy/README.md
```

```
echo "" | sudo tee -a /var/www/caddy/README.md
```

```
echo "Este servidor está funcionando correctamente." | sudo tee -a /var/www/caddy/README.md
```

```
echo "" | sudo tee -a /var/www/caddy/README.md
```

```
echo "## Características" | sudo tee -a /var/www/caddy/README.md
```

```
echo "- Servidor moderno" | sudo tee -a /var/www/caddy/README.md
```

```
echo "- HTTPS automático" | sudo tee -a /var/www/caddy/README.md
```

```
echo "- Fácil configuración" | sudo tee -a /var/www/caddy/README.md
```

```
E: unable to locate package update
ubuntu@ip-172-31-76-39:~$ sudo mkdir -p /var/www/caddy
ubuntu@ip-172-31-76-39:~$ echo "# Bienvenido a Caddy" | sudo tee /var/www/caddy/README.md
# Bienvenido a Caddy
ubuntu@ip-172-31-76-39:~$ echo "" | sudo tee -a /var/www/caddy/README.md

ubuntu@ip-172-31-76-39:~$ echo "Este servidor está funcionando correctamente." | sudo tee -a /var/www/caddy/README.md
Este servidor está funcionando correctamente.
ubuntu@ip-172-31-76-39:~$ echo "" | sudo tee -a /var/www/caddy/README.md

ubuntu@ip-172-31-76-39:~$ echo "## Características" | sudo tee -a /var/www/caddy/README.md
## Características
ubuntu@ip-172-31-76-39:~$ echo "- Servidor moderno" | sudo tee -a /var/www/caddy/README.md
- Servidor moderno
ubuntu@ip-172-31-76-39:~$ echo "- HTTPS automático" | sudo tee -a /var/www/caddy/README.md
- HTTPS automático
ubuntu@ip-172-31-76-39:~$ echo "- Fácil configuración" | sudo tee -a /var/www/caddy/README.md
- Fácil configuración
ubuntu@ip-172-31-76-39:~$
```

Se crea un archivo `README.md` con contenido Markdown para validar que Caddy sirve correctamente archivos en formato texto.

Ver el el contenido del archivo

cat /var/www/caddy/README.md

```
ubuntu@ip-172-31-76-39:~$ cat /var/www/caddy/README.md
# Bienvenido a Caddy

Este servidor está funcionando correctamente.

## Características
- Servidor moderno
- HTTPS automático
- Fácil configuración
ubuntu@ip-172-31-76-39:~$
```

Contenido del archivo

Configuración personalizada del Caddyfile

sudo nano /etc/caddy/Caddyfile


```
# domain name.

:8082 {

    # Set this path to your site's directory.
    root * /usr/share/caddy

    # Enable the static file server.
    file_server

    # Another common task is to set up a reverse proxy:
    # reverse_proxy localhost:8080

    # Or serve a PHP site through php-fpm:
    # php_fastcgi localhost:9000
}

# Refer to the Caddy docs for more information:
# https://caddyserver.com/docs/caddyfile
```

Modificamos este archivo para que quede como en la siguiente captura

```
# Unless the file starts with a global options block, the first
# uncommented line is always the address of your site.
#
# To use your own domain name (with automatic HTTPS), first make
# sure your domain's A/AAAA DNS records are properly pointed to
# this machine's public IP, then replace ":80" below with your
# domain name.
:8082 {
    root * /var/www/caddy
    file_server browse

    @markdown path *.md
    header @markdown Content-Type text/plain
}
```

Esta es la configuración que debemos dejar

Descargamos la imagen

Ejecutamos `curl -o /tmp/test-image.jpg https://www.python.org/static/apple-touch-icon-144x144-precomposed.png`

Mover la imagen a la carpeta servida por Caddy

`sudo mv /tmp/test-image.jpg /var/www/caddy/test.jpg`



Caddy sirviendo archivos estáticos

Instalar Certbot + módulo de Apache

`sudo apt install certbot python3-certbot-apache -y`

Crear certificado autofirmado

`sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \`

`-keyout /etc/ssl/private/apache-selfsigned.key \`

`-out /etc/ssl/certs/apache-selfsigned.crt`

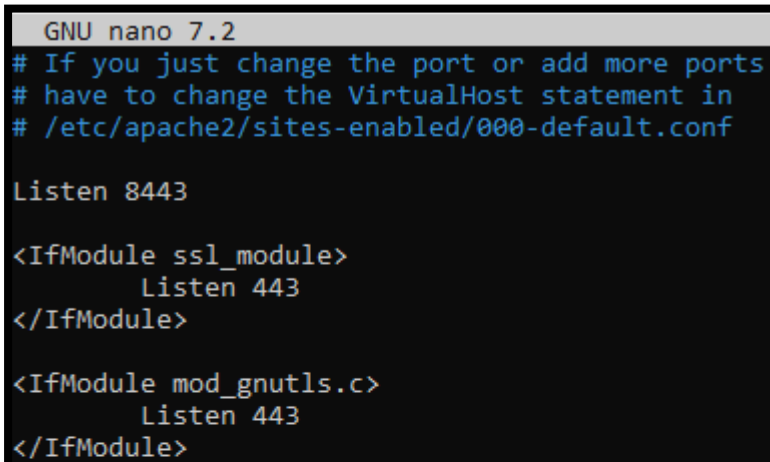
```
ubuntu@ip-172-31-76-39:~$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/apache-selfsigned.key \
-out /etc/ssl/certs/apache-selfsigned.crt
.....
.....
.....
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank.
For some fields there will be a default value,
if you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:ES
State or Province Name (full name) [Some-State]:Madrid
Locality Name (eg, city) []:Madrid
Organization Name (eg, company) [Internet Widgits Pty Ltd]:kike
Organizational Unit Name (eg, section) []:kike
Common Name (e.g. server FQDN or YOUR name) []:kike2
Email Address []:kike@gmail.com
ubuntu@ip-172-31-76-39:~$
```

Habilitar el módulo SSL en Apache

`sudo a2enmod ssl`

Cambiar el puerto SSL a 8443

`sudo nano /etc/apache2/ports.conf`



```
GNU nano 7.2
# If you just change the port or add more ports
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf

Listen 8443

<IfModule ssl_module>
    Listen 443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 443
</IfModule>
```

Modificar el VirtualHost SSL para usar 8443 y tu certificado

`sudo nano /etc/apache2/sites-available/default-ssl.conf`

```

GNU nano 7.2 /etc/apache2/sites-available/000-default.conf
<VirtualHost *:8443>
    ServerAdmin webmaster@localhost

    DocumentRoot /var/www/html

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf

    #
    # SSL Engine Switch:
    # Enable/Disable SSL for this virtual host.
    SSLEngine on

    #
    # A self-signed (snakeoil) certificate can be created by installing
    # the ssl-cert package. See
    # /usr/share/doc/apache2/README.Debian.gz for more info.
    # If both key and certificate are stored in the same file, only the
    # SSLCertificateFile directive is needed.
    SSLCertificateFile /etc/ssl/certs/ssl-cert-snakeoil.pem
    SSLCertificateKeyFile /etc/ssl/private/ssl-cert-snakeoil.key

    #
    # Server Certificate Chain:
    # Point SSLCertificateChainFile at a file containing the
    # concatenation of PEM encoded CA certificates which form the
    # certificate chain for the server certificate. Alternatively
    # the referenced file can be the same as SSLCertificateFile
    # when the CA certificates are directly appended to the server
    # certificate for convinience.
    #SSLCertificateChainFile /etc/apache2/ssl.crt/server-ca.crt

    #
    # Certificate Authority (CA):
    # Set the CA certificate verification path where to find CA
    # certificates for client authentication or alternatively one
    # huge file containing all of them (file must be PEM encoded)
    # Note: Inside SSLCACertificatePath you need hash symlinks
    # to point to the certificate files. Use the provided
    # Makefile to update the hash symlinks after changes.
    #SSLCACertificatePath /etc/ssl/certs/
    #SSLCACertificateFile /etc/apache2/ssl.crt/ca-bundle.crt

    #
    # Certificate Revocation Lists (CRL):
    # Set the CA revocation path where to find CA CRLs for client
    # authentication.

```

Cambiamos el 443 por el 8443

Ahora tenemos que cambiar los certificados en el archivo de configuración:

```

ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined

# For most configuration files from conf-available/, which are
# enabled or disabled at a global level, it is possible to
# include a line for only one particular virtual host. For example the
# following line enables the CGI configuration for this host only
# after it has been globally disabled with "a2disconf".
#Include conf-available/serve-cgi-bin.conf

# SSL Engine Switch:
# Enable/Disable SSL for this virtual host.
SSLEngine on

# A self-signed (snakeoil) certificate can be created by installing
# the ssl-cert package. See
# /usr/share/doc/apache2/README.Debian.gz for more info.
# If both key and certificate are stored in the same file, only the
# SSLCertificateFile directive is needed.
SSLCertificateFile /etc/ssl/certs/apache-selfsigned.crt
SSLCertificateKeyFile /etc/ssl/private/apache-selfsigned.key

```

Certificados correctos



Apache corriendo en el 8443

Parte 5 de la Segunda práctica: verificación final

Vamos a comprobar el estado de todos los servicios

```
ubuntu@ip-172-31-76-39:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-11-20 08:18:49 UTC; 46min ago
     Docs: https://httpd.apache.org/docs/2.4/
  Process: 520 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
  Process: 939 ExecReload=/usr/sbin/apachectl graceful (code=exited, status=0/SUCCESS)
 Main PID: 559 (apache2)
    Tasks: 55 (limit: 1017)
   Memory: 9.6M (peak: 12.2M)
      CPU: 258ms
   CGroup: /system.slice/apache2.service
           └─559 /usr/sbin/apache2 -k start
             └─978 /usr/sbin/apache2 -k start
               └─979 /usr/sbin/apache2 -k start

Nov 20 08:18:49 ip-172-31-76-39 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Nov 20 08:18:49 ip-172-31-76-39 systemd[1]: Started apache2.service - The Apache HTTP Server.
Nov 20 08:18:50 ip-172-31-76-39 systemd[1]: Reloading apache2.service - The Apache HTTP Server...
Nov 20 08:18:50 ip-172-31-76-39 systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
ubuntu@ip-172-31-76-39:~$
```

Apache funcionando correctamente

```
ubuntu@ip-172-31-76-39:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-11-20 08:18:49 UTC; 47min ago
     Docs: man:nginx(8)
  Process: 551 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Process: 598 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
 Main PID: 621 (nginx)
    Tasks: 3 (limit: 1017)
   Memory: 3.7M (peak: 3.9M)
      CPU: 19ms
   CGroup: /system.slice/nginx.service
           └─621 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─622 "nginx: worker process"
               └─623 "nginx: worker process"

Nov 20 08:18:49 ip-172-31-76-39 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Nov 20 08:18:49 ip-172-31-76-39 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-76-39:~$
```

Gnix funcionando correctamente

```

ubuntu@ip-172-31-76-39:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-11-20 08:18:49 UTC; 47min ago
     Docs: man:nginx(8)
   Process: 551 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Process: 598 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Main PID: 621 (nginx)
    Tasks: 3 (limit: 1017)
   Memory: 3.7M (peak: 3.9M)
      CPU: 19ms
   CGroup: /system.slice/nginx.service
           └─621 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─622 "nginx: worker process"
               └─623 "nginx: worker process"

Nov 20 08:18:49 ip-172-31-76-39 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Nov 20 08:18:49 ip-172-31-76-39 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-76-39:~$ sudo netstat -tulpn | grep caddy
sudo: netstat: command not found
ubuntu@ip-172-31-76-39:~$

```

Caddy funcionando correctamente

Ahora vamos a probar los puertos que sirve cada servicio:

```

└─622 "nginx: worker process"
└─623 "nginx: worker process"

Nov 20 08:18:49 ip-172-31-76-39 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Nov 20 08:18:49 ip-172-31-76-39 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-76-39:~$ sudo netstat -tulpn | grep caddy
sudo: netstat: command not found
ubuntu@ip-172-31-76-39:~$ sudo netstat -tulpn | grep apache2
sudo: netstat: command not found
ubuntu@ip-172-31-76-39:~$ sudo netstat -tulpn | grep apache2
sudo: netstat: command not found
ubuntu@ip-172-31-76-39:~$ sudo netstat -tulpn | grep apache2
sudo: netstat: command not found
ubuntu@ip-172-31-76-39:~$ sudo ss -tulpn | grep apache2
tcp        LISTEN     0      511                *:*                *:*                users:((("apache2",pid=979,fd=4),("apache2",pid=978,fd=4),("apache2",pid=559,fd=4)))
tcp        LISTEN     0      511                *:443              *:443              users:((("apache2",pid=979,fd=6),("apache2",pid=978,fd=6),("apache2",pid=559,fd=6)))
ubuntu@ip-172-31-76-39:~$ sudo ss -tulpn | grep nginx
tcp        LISTEN     0      511                0.0.0.0:*          0.0.0.0:*          users:((("nginx",pid=623,fd=5),("nginx",pid=622,fd=5),("nginx",pid=621,fd=5)))
tcp        LISTEN     0      511                [::]:8081          [::]:8081          users:((("nginx",pid=623,fd=6),("nginx",pid=622,fd=6),("nginx",pid=621,fd=6)))
ubuntu@ip-172-31-76-39:~$ sudo ss -tulpn | grep caddy
tcp        LISTEN     0      4096             127.0.0.1:2010     0.0.0.0:*          users:((("caddy",pid=522,fd=3)))
tcp        LISTEN     0      4096                *:8082             *:8082             users:((("caddy",pid=522,fd=7)))
ubuntu@ip-172-31-76-39:~$

```

Puertos que escucha cada servicio

CONCLUSION FINAL

A lo largo de este trabajo hemos podido experimentar de manera práctica cómo se administra un servidor real en la nube y cómo se trasladan conocimientos locales a un entorno profesional como AWS. No solo hemos aprendido a desplegar una instancia EC2 y acceder a ella mediante SSH utilizando claves PEM —lo cual representa un pilar fundamental de la seguridad en sistemas distribuidos— sino que también hemos comprobado la importancia de configurar correctamente los grupos de seguridad, los puertos y los servicios que un servidor expone al exterior.

Replicar la segunda práctica dentro de un servidor remoto nos ha permitido comprender que los procedimientos técnicos no cambian, pero sí cambia el contexto y la responsabilidad: cada acción realizada sobre una instancia en la nube es una decisión de administración real. Esta experiencia hace evidente la diferencia entre trabajar en local y gestionar un recurso remoto que depende de una infraestructura global. También refuerza la idea de que la nube no es un concepto abstracto, sino un entorno tangible donde cada configuración tiene un impacto directo en la disponibilidad, seguridad y accesibilidad del servicio.

En definitiva, este trabajo no solo nos ha permitido poner en práctica comandos y configuraciones, sino también reflexionar sobre el rol del administrador de sistemas en un mundo cada vez más orientado a la virtualización y al trabajo remoto. Hemos aprendido a trabajar con responsabilidad, precisión y metodología, entendiendo que la nube es hoy uno de los pilares fundamentales en la administración moderna de sistemas. Esta práctica nos deja una base sólida para afrontar entornos profesionales donde la gestión remota, la automatización y la seguridad son esenciales.

